

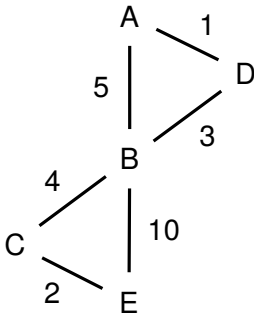
Single-Source Shortest Paths

Comp Sci 214, Spring 2025

Don't forget to check in on Youhere!

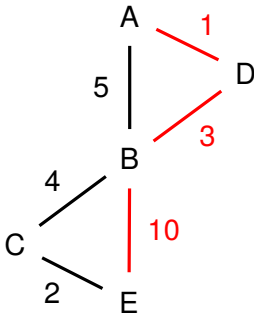
The Problem: Finding Shortest Paths

On this weighted graph, what is the shortest path from A to E?



The Problem: Finding Shortest Paths

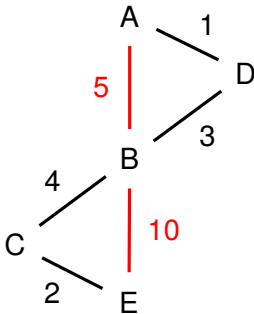
On this weighted graph, what is the shortest path from A to E?



A-D-B-E: 14

The Problem: Finding Shortest Paths

On this weighted graph, what is the shortest path from A to E?

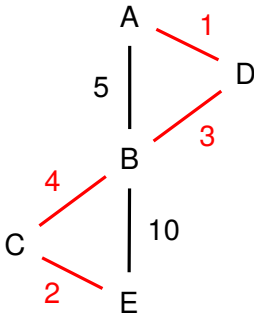


A-D-B-E: 14

A-B-E: 15. Worse!

The Problem: Finding Shortest Paths

On this weighted graph, what is the shortest path from A to E?



A-D-B-E: 14

A-B-E: 15. Worse!

A-D-B-C-E: 10. We have a winner!

The Problem: Finding Shortest Paths

Why?

- Directions in a map application
- Network routing
- VLSI circuit design
- Etc.

Typically generalized as single-source shortest paths (SSSP):

- Find the shortest path to everywhere from a given vertex.

Why generalize?

- To find for one destination, we may end up doing all the work needed to find all of them anyway.
- So might as well.

Applies to directed and undirected graphs alike.

Useful on weighted graphs; otherwise BFS suffices.

The Solution: Just Relax

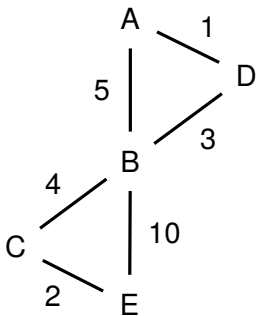
Relaxation:

- At the start, maximally overestimate costs.
- Then bring estimate down as you learn more information.
 - ▶ We call this step the *relaxation* step.
- Once you've relaxed enough, you have the real cost.
 - ▶ For some definition of "enough".

General approach to algorithms, applies to all sorts of problems:

- Reinforcement learning
- Graphics rendering
- etc.

Relaxation for SSSP

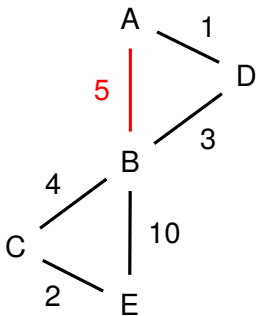


Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

To start, most pessimistic costs possible: everything ∞ far

Relaxation for SSSP



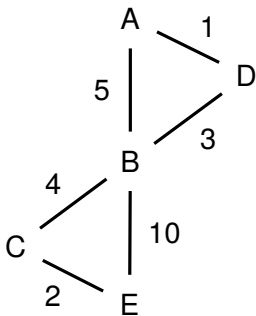
Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	∞	
D	∞	
E	∞	

As we look at (relax) edges, we learn more information.

E.g., we can get to B in $0 + 5$ by going through A!
(Best known distance to A, plus weight of edge A–B.)

Relaxation for SSSP



Best paths found so far:

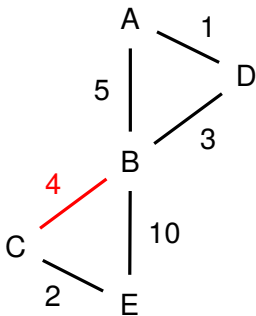
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	∞	
D	∞	
E	∞	

How did we know to pick A–B? We didn't!

Right now, picking arbitrarily to demonstrate relaxation.

Actual SSSP algorithms will have specific criteria for picking.

Relaxation for SSSP



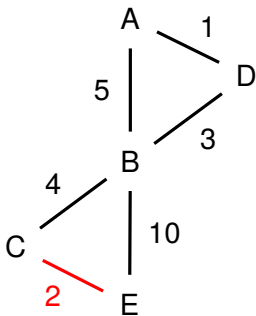
Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	9	B
D	∞	
E	∞	

If we look at more edges, we can learn more.

We can get to C in $5 + 4 = 9$ by going through B.

Relaxation for SSSP

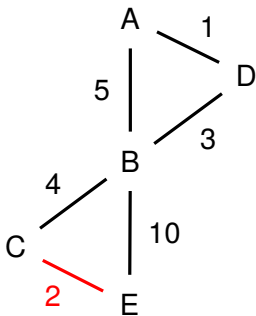


Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	9	B
D	∞	
E	∞	

We can get to E in ??? by going through ???.

Relaxation for SSSP

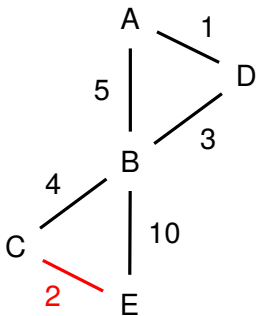


Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	9	B
D	∞	
E	11	

We can get to E in $9 + 2 = 11$ by going through ???.

Relaxation for SSSP

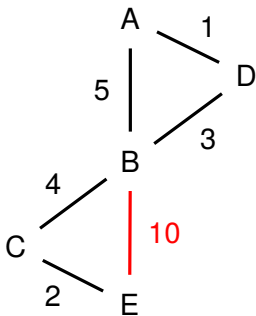


Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	9	B
D	∞	
E	11	C

We can get to E in $9 + 2 = 11$ by going through C.

Relaxation for SSSP



Best paths found so far:

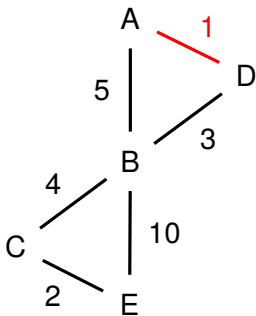
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	9	B
D	∞	
E	11	C

Some edges, however, don't give us any useful information.

We could get to E in 15 via B, but worse than the 11 we know. Not interested.

We call this an *unproductive* relaxation.

Relaxation for SSSP

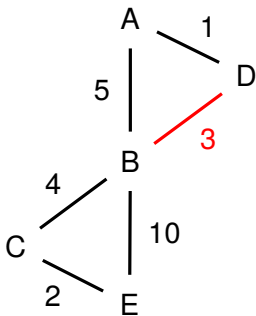


Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	5	A
C	9	B
D	1	A
E	11	C

We can get to D in $0 + 1$ by going through A.

Relaxation for SSSP



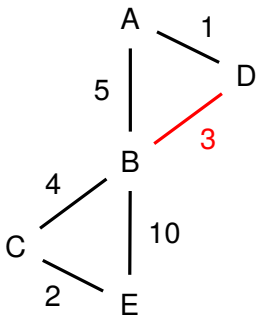
Best paths found so far:

v	dist[v]	pred[v]
A	0	
B	5	A
C	9	B
D	1	A
E	11	C

We found a better path to B! $1 + 3 < 5$.

Let's update our table for B.

Relaxation for SSSP



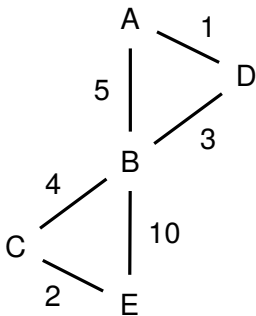
Best paths found so far:

v	dist[v]	pred[v]
A	0	
B	4	D
C	9	B
D	1	A
E	11	C

We found a better path to B! $1 + 3 < 5$.

Let's update our table for B.

Relaxation for SSSP



Best paths found so far:

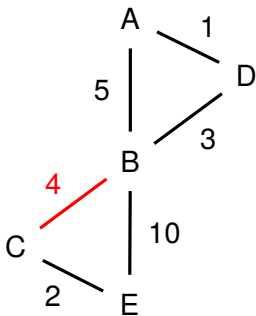
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	4	D
C	9	B
D	1	A
E	11	C

Oh, but our graph is undirected! $D-B = B-D$!

Did we find a shorter path to D too? No. $5 + 3 > 1$.

We needed to check all along. But we got away with it (∞).

Relaxation for SSSP



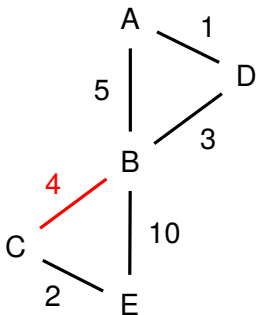
Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	4	D
C	9	B
D	1	A
E	11	C

Now that we have a better path to B, if we look at B–C again, we also find a better path! $4 + 4 < 9$

This means we may need to relax each edge multiple times!

Relaxation for SSSP



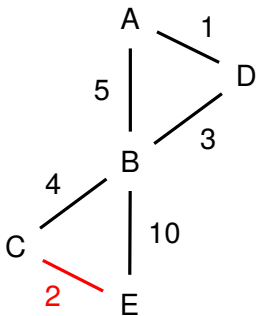
Best paths found so far:

v	dist[v]	pred[v]
A	0	
B	4	D
C	8	B
D	1	A
E	11	C

Now that we have a better path to B, if we look at B–C again, we also find a better path! $4 + 4 < 9$

This means we may need to relax each edge multiple times!

Relaxation for SSSP



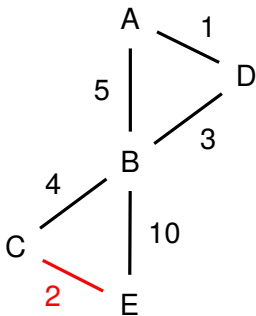
Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	4	D
C	8	B
D	1	A
E	11	C

Same for C-E! $8 + 2 < 11$

Every time we relax an edge, we're "pumping" information forward one step.

Relaxation for SSSP



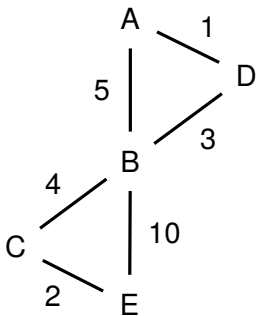
Best paths found so far:

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	4	D
C	8	B
D	1	A
E	10	C

Same for C-E! $8 + 2 < 11$

Every time we relax an edge, we're "pumping" information forward one step.

Relaxation for SSSP



Best paths found so far:

v	dist[v]	pred[v]
A	0	
B	4	D
C	8	B
D	1	A
E	10	C

Are we done? Maybe!

Different algorithms have different stopping conditions.

But they're all guaranteed to find the shortest path to each node!

For our relaxation demonstration, we'll stop here.

Stop! Question time!

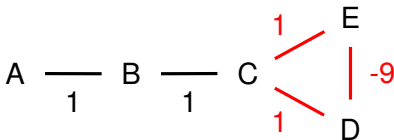
Before we move on to algorithms that use relaxation...

Anything unclear so far?

Anything I missed?

A note on negative cycles

SSSP doesn't make sense for all graphs.



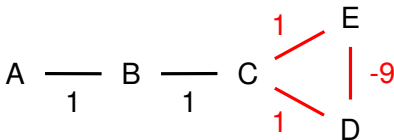
A-B-C-E has length 3.

A-B-C-D-E has length -6. Shorter!

A-B-C-D-E-C-D-E has length -13. Even shorter!

A note on negative cycles

SSSP doesn't make sense for all graphs.



A-B-C-E has length 3.

A-B-C-D-E has length -6. Shorter!

A-B-C-D-E-C-D-E has length -13. Even shorter!

Could keep going forever...

C-D-E-C is a *negative cycle*: negative total weight (-7)

SSSP doesn't make sense for graphs with negative cycles:
no shortest paths!

The Bellman-Ford algorithm

Solves	SSSP for graphs with no negative cycles
Main idea	Relax every edge $ V - 1$ times
Time complexity	$\mathcal{O}(V \times E)$

- Pick edges in arbitrary order (w.e. `get_all_edges` returns)
- Relax each edge $|V| - 1$ times, then we're done

Intuition

- Each relaxation propagates information 1 edge along path
- After 1 iteration, found all shortest paths of length 1
 - ▶ No guarantees about the others; arbitrary relaxation order
- After n iterations, found all shortest paths of length $\leq n$

Essentially brute force relaxation

- Relax until problem is solved
- Don't even try to be smart about it

The Bellman-Ford algorithm

Solves	SSSP for graphs with no negative cycles
Main idea	Relax every edge $ V - 1$ times
Time complexity	$\mathcal{O}(V \times E)$

Why is $|V| - 1$ times *sufficient*?



- Each relaxation propagates information 1 edge along path
- How many edges does the shortest (i.e., *least weight*) path with the *most edges* have in the worst case? $|V| - 1$
- Any path longer than $|V| - 1$ edges has a cycle!
 - ▶ Why? We see at least one node more than once!
- No neg. cycles $\rightarrow$ can cut out cycle and get shorter path!

The Bellman-Ford algorithm

Solves	SSSP for graphs with no negative cycles
Main idea	Relax every edge $ V - 1$ times
Time complexity	$\mathcal{O}(V \times E)$

Why is $|V| - 1$ times *necessary*?



- Say we're looking for shortest path between A and E.
- Bellman-Ford relaxes edges in an *arbitrary* order (i.e., could be arbitrarily bad!)
- Worst case, relax edges in the order: E-D, C-D, B-C, A-B
- Need the full $|V| - 1$ iterations!

The Bellman-Ford algorithm

Input: A graph *graph* and a starting vertex *start*

Output: Tables of vertex distances *dist* and predecessors *pred*

Initialize the table.

for every vertex *v* in *graph* do

$dist[v] \leftarrow \infty;$

$pred[v] \leftarrow \text{None};$

end

$dist[start] \leftarrow 0;$

for $|Vertices(\textit{graph})| - 1$ iterations do

 for every edge (*v,u*) with weight *w* in *graph* do

This conditional relaxes edge v-u.

 if $dist[v] + w < dist[u]$ then

(If undirected, we'd also relax edge u-v.)

$dist[u] \leftarrow dist[v] + w;$

$pred[u] \leftarrow v;$

 end

 end

end

Bellman-Ford, with checking

If we have no negative cycles, we have the answer.

But we can *check* for negative cycles, rather than *assuming* there are none.

After we're done, relax all the edges one more time.

If anything improves, we have a negative cycle!

```
for every edge  $(v,u)$  with weight  $w$  in graph do
|   if  $dist[v] + w < dist[u]$  then
|       graph contains a negative cycle!
|   end
end
```


The issue with Bellman-Ford

$\mathcal{O}(|V| \times |E|)$ is not great.

- Because $|E|$ is $\mathcal{O}(|V|^2)$ in the worst case, that's $\mathcal{O}(|V|^3)$
- Cubic, yikes!

There are two main sources of waste:

- We may not need the full $|V| - 1$ relaxations for each edge!
 - ▶ Could stop when a round of relaxing all edges doesn't improve anything.
 - ▶ Called a *fixed point* algorithm. Comes up a lot in program analysis (compilers, security).
- Relaxing edges in an arbitrary order may not be optimal!
 - ▶ See earlier example (straight line graph)

Next algorithm picks a clever order, relaxes each edge *once*!

Stop! Question time!

Before we move on to today's second algorithm...

Anything unclear so far?

Anything I missed?

Dijkstra's algorithm

Dijkstra's algorithm

Named after Edsger W. Dijkstra (pronounced: *dike-strah*).

- Early CS pioneer, 1972 Turing award
- A name you'll see often in CS (so learn to pronounce it!)

Dijkstra's algorithm

Named after Edsger W. Dijkstra (pronounced: *dike-strah*).

- Early CS pioneer, 1972 Turing award
- A name you'll see often in CS (so learn to pronounce it!)

Solves	SSSP for graphs with no negative <i>edges</i>
Main idea	Relax the edges in a clever order
Time complexity	$\mathcal{O}(V ^2 \log V)$

- More restrictive requirements
- But we'll be able to get better complexity

Dijkstra's algorithm

Named after Edsger W. Dijkstra (pronounced: *dike-strah*).

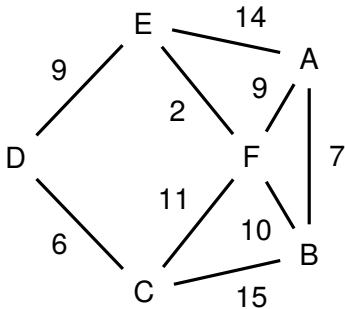
- Early CS pioneer, 1972 Turing award
- A name you'll see often in CS (so learn to pronounce it!)

Solves	SSSP for graphs with no negative <i>edges</i>
Main idea	Relax the edges in a clever order
Time complexity	$\mathcal{O}(V ^2 \log V)$

- More restrictive requirements
- But we'll be able to get better complexity
- What's the clever order?
 - ▶ Relax the edges coming out of the *nearest* vertex so far
 - ▶ Then repeat with the next nearest, etc.
 - ▶ Reminiscent of BFS, but with weights!
 - ▶ Will be able to relax each edge only once!

Dijkstra's algorithm demonstration

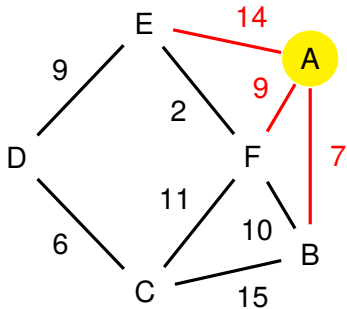
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	



done: {}

Dijkstra's algorithm demonstration

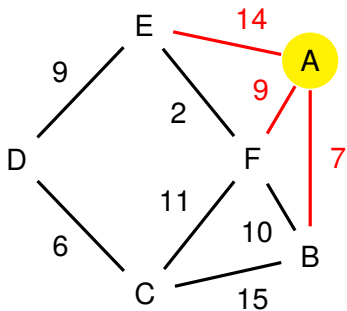
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	



done: {}

Dijkstra's algorithm demonstration

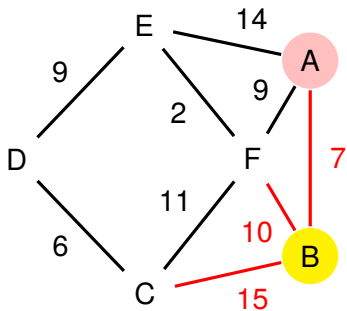
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	∞	
D	∞	
E	14	A
F	9	A



done: {A}

Dijkstra's algorithm demonstration

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	∞	
D	∞	
E	14	A
F	9	A



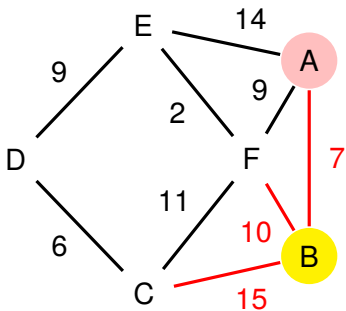
done: {A}

When we get to B, we **know** we found the shortest path to it.

B was nearest. Any other path would need to go through a further node!

Dijkstra's algorithm demonstration

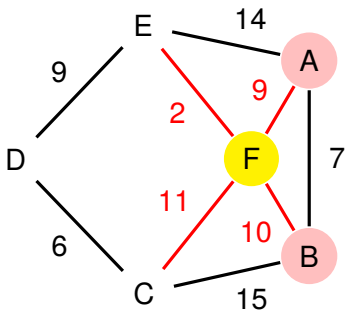
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	22	B
D	∞	
E	14	A
F	9	A



done: {A, B}

Dijkstra's algorithm demonstration

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	22	B
D	∞	
E	14	A
F	9	A



done: $\{A, B\}$

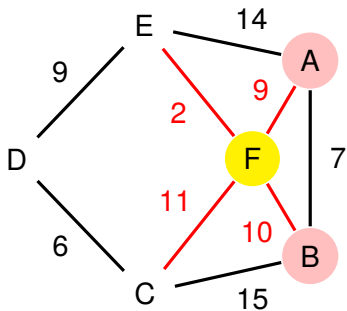
When we get to F, we **know** we found the shortest path to it.

Same reasoning as with B.

Also, just like with BFS!

Dijkstra's algorithm demonstration

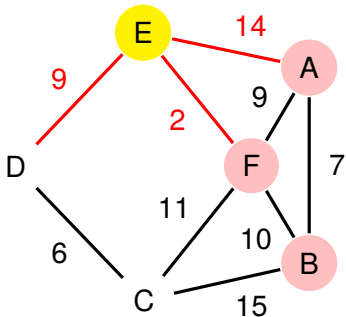
v	dist[v]	pred[v]
A	0	
B	7	A
C	20	F
D	∞	
E	11	F
F	9	A



done: {A, B, F}

Dijkstra's algorithm demonstration

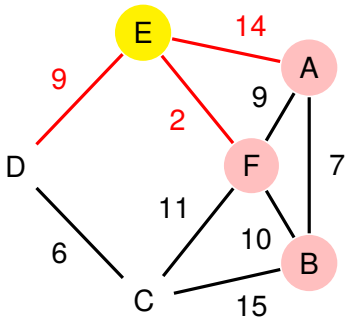
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	∞	
E	11	F
F	9	A



done: {A, B, F}

Dijkstra's algorithm demonstration

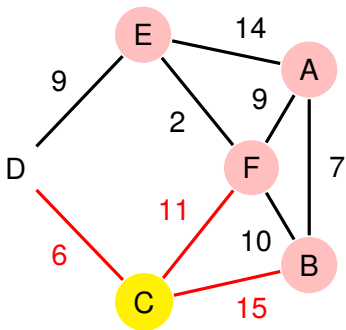
v	dist[v]	pred[v]
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A



done: {A, B, E, F}

Dijkstra's algorithm demonstration

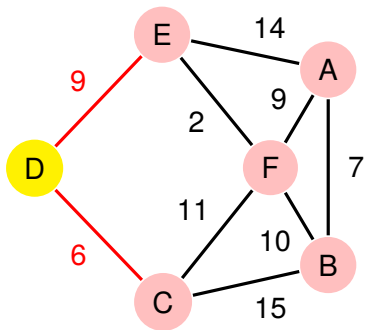
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A



done: $\{A, B, E, F\}$

Dijkstra's algorithm demonstration

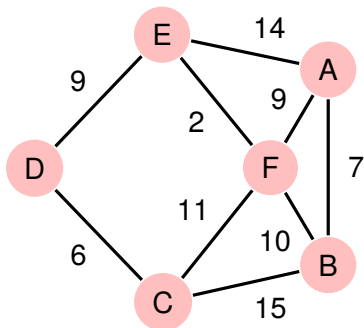
v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A



done: $\{A, B, C, E, F\}$

Dijkstra's algorithm demonstration

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A



done: $\{A, B, C, D, E, F\}$

We found all the shortest paths.

QUIZ: Shortest path from A to D?

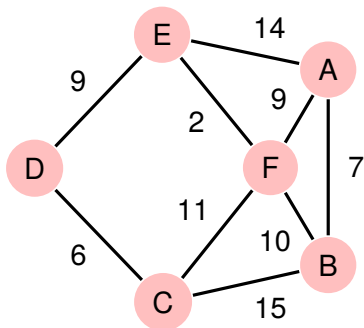
A: A-E-D

B: A-F-E-D

C: A-B-C-D

Dijkstra's algorithm demonstration

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A



done: $\{A, B, C, D, E, F\}$

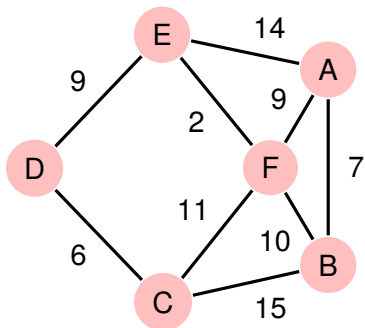
We found all the shortest paths.

QUIZ: Shortest path from A to D?

A: A-E-D **B:** A-F-E-D **C:** A-B-C-D

Dijkstra's algorithm demonstration

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A

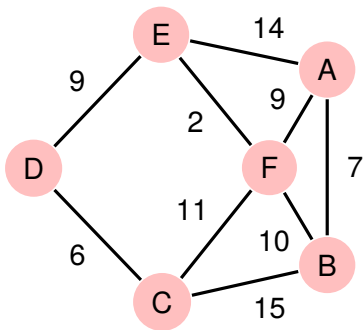


done: $\{A, B, C, D, E, F\}$

Q: What ADT would you use to get the nearest node at each step?

Dijkstra's algorithm demonstration

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A



done: $\{A, B, C, D, E, F\}$

Q: What ADT would you use to get the nearest node at each step?

Priority queue!

Dijkstra's algorithm (1/2)

Input: A graph *graph* and a starting vertex *start*

Output: Tables of vertex distances *dist* and predecessors *pred*

for every vertex *v* **in** *graph* **do**

 | $dist[v] \leftarrow \infty$; $pred[v] \leftarrow \text{None}$;

end

$dist[start] \leftarrow 0$;

lower priority = shorter distance!

might need to hold up to $|E|$ elements

todo $\leftarrow$ empty priority queue;

keep track of nodes we finished, to not do them again

done $\leftarrow$ empty vertex set;

at first, only the starting vertex in the priority queue $\rightarrow$ start there

start is at distance 0 from start

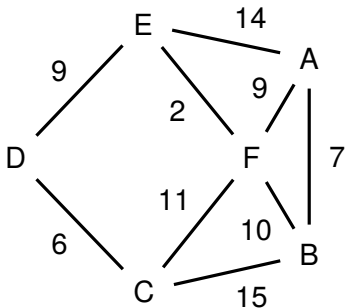
Insert(*todo*, 0, *start*);

Dijkstra's algorithm (2/2)

```
while todo is not empty do
    # pick the nearest vertex
     $v \leftarrow \text{RemoveMin}(\textit{todo})$ ;
    if  $v \notin \textit{done}$  then
         $\textit{done} \leftarrow \textit{done} \cup \{v\}$ ;
        for every outgoing edge  $(v,u)$  with weight  $w$  do
            # relax outgoing edges
            if  $\textit{dist}[v] + w < \textit{dist}[u]$  then
                 $\textit{dist}[u] \leftarrow \textit{dist}[v] + w$ ;
                 $\textit{pred}[u] \leftarrow v$ ;
                # add neighbor to priority queue
                Insert( $\textit{todo}, \textit{dist}[u], u$ )
            end
        end
    end
end
end
```

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

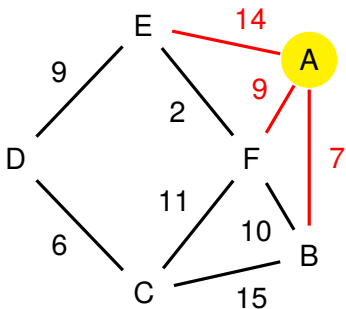


done: {}

PQ: $\langle 0 : A \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

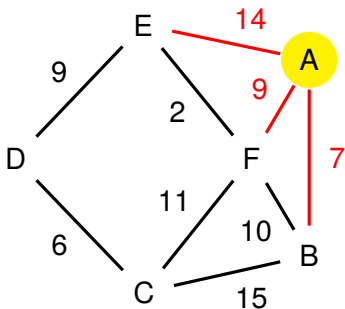


done: {}

PQ: $\langle \langle \rangle \rangle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	∞	
D	∞	
E	14	A
F	9	A

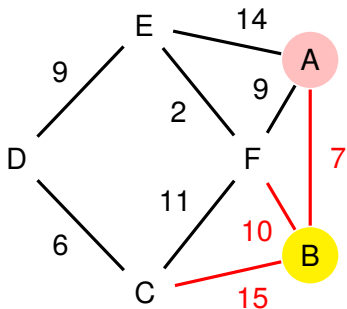


done: {A}

PQ: $\langle 7:B, 9:F, 14:E \rangle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	∞	
D	∞	
E	14	A
F	9	A

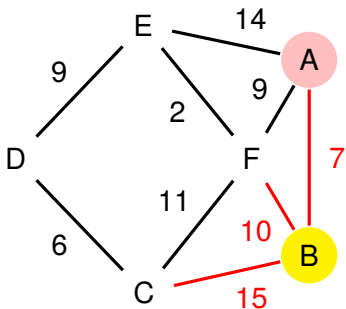


done: {A}

PQ: $\langle 9 : F, 14 : E \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	22	B
D	∞	
E	14	A
F	9	A

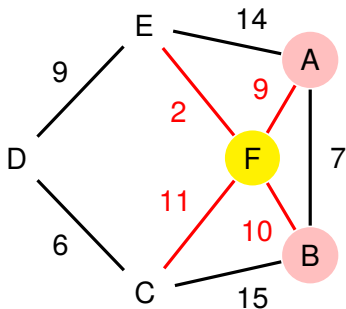


done: {A, **B**}

PQ: $\langle 9 : F, 14 : E, 22 : \text{C} \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	22	B
D	∞	
E	14	A
F	9	A

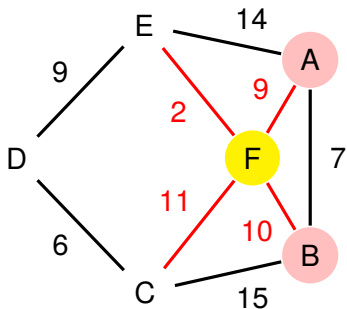


done: $\{A, B\}$

PQ: $\langle 14 : E, 22 : C \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	∞	
E	11	F
F	9	A

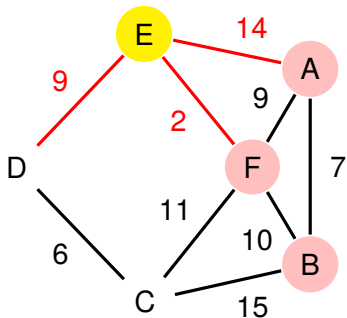


done: {A, B, F}

PQ: $\langle 11 : E, 14 : E, 20 : C, 22 : C \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	∞	
E	11	F
F	9	A

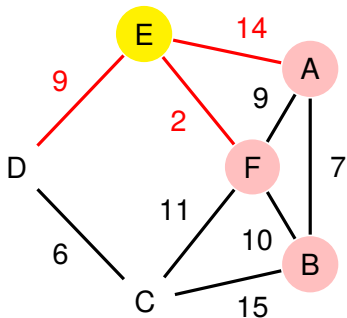


done: $\{A, B, F\}$

PQ: $\langle 14 : E, 20 : C, 22 : C \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A

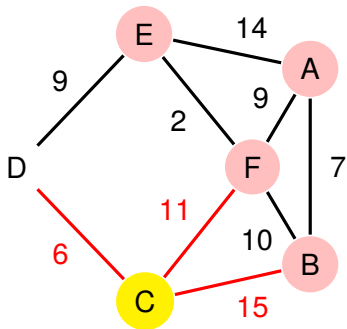


done: {A, B, E, F}

PQ: $\langle 14 : E, 20 : C, 20 : D, 22 : C \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A

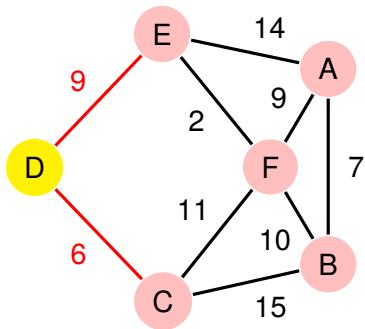


done: $\{A, B, E, F\}$

PQ: $\langle 20 : D, 22 : C \rangle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A

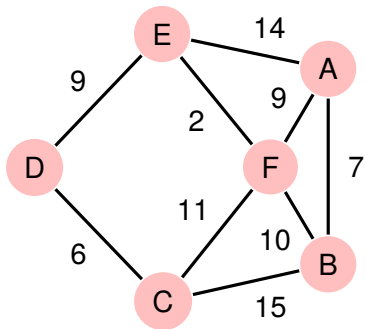


done: {A, B, C, E, F}

PQ: $\langle 22 : C \langle$

Dijkstra's algorithm with priority queue

v	$\text{dist}[v]$	$\text{pred}[v]$
A	0	
B	7	A
C	20	F
D	20	E
E	11	F
F	9	A



done: {A, B, C, **D**, E, F}

PQ: $\langle \langle \rangle \rangle$

Time Complexity of Dijkstra's algorithm

- Relax every edge once: $\mathcal{O}(|E|)$

Time Complexity of Dijkstra's algorithm

- Relax every edge once: $\mathcal{O}(|E|)$
- For every edge, we (might) do an insert
 - ▶ $\rightarrow$ in the worst case, priority queue of size $|E|$
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap

Time Complexity of Dijkstra's algorithm

- Relax every edge once: $\mathcal{O}(|E|)$
- For every edge, we (might) do an insert
 - ▶ $\rightarrow$ in the worst case, priority queue of size $|E|$
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap
- For every edge, we (might) do a `remove_min`
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap

Time Complexity of Dijkstra's algorithm

- Relax every edge once: $\mathcal{O}(|E|)$
- For every edge, we (might) do an insert
 - ▶ $\rightarrow$ in the worst case, priority queue of size $|E|$
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap
- For every edge, we (might) do a `remove_min`
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap
- So Dijkstra's algorithm is
$$\mathcal{O}\left(|E| \times \mathcal{O}(\log |E|) + |E| \times \mathcal{O}(\log |E|)\right)$$

Time Complexity of Dijkstra's algorithm

- Relax every edge once: $\mathcal{O}(|E|)$
- For every edge, we (might) do an insert
 - ▶ $\rightarrow$ in the worst case, priority queue of size $|E|$
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap
- For every edge, we (might) do a remove_min
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap
- So Dijkstra's algorithm is
$$\mathcal{O}\left(|E| \times \mathcal{O}(\log |E|) + |E| \times \mathcal{O}(\log |E|)\right) = \mathcal{O}\left(|E| \log |E|\right)$$

Time Complexity of Dijkstra's algorithm

- Relax every edge once: $\mathcal{O}(|E|)$
- For every edge, we (might) do an insert
 - ▶ $\rightarrow$ in the worst case, priority queue of size $|E|$
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap
- For every edge, we (might) do a `remove_min`
 - ▶ $\mathcal{O}(\log |E|)$ with a binary heap
- So Dijkstra's algorithm is
$$\mathcal{O}\left(|E| \times \mathcal{O}(\log |E|) + |E| \times \mathcal{O}(\log |E|)\right) = \mathcal{O}\left(|E| \log |E|\right)$$
- Because $\mathcal{O}(|E|)$ is bounded by $\mathcal{O}(|V|^2)$, bounded by
$$\mathcal{O}\left(|V|^2 \log(|V|^2)\right) = \mathcal{O}\left(|V|^2 \times 2 \log |V|\right) = \mathcal{O}\left(|V|^2 \log |V|\right)$$

Nicer than Bellman-Ford's $\mathcal{O}(|V|^3)$!

See Dijkstra in Action

- <https://www.cs.usfca.edu/~galles/visualization/Dijkstra.html>

Next time: Another graph
algorithm and another ADT
to go with it